

AI에게 어셈 코드 작성을 시켜보았다.

유영천

<https://megayuchi.com>

발단

- 신규 강의에 사용할 예제에서 ray와 삼각형의 교차 테스트를 수행하는 IntersectTriangle()함수를 사용.
- 드라마틱한 성능개선을 보여줄 계획 - 화면 픽셀 수 만큼 호출될 것이므로 성능 차이가 크게 나타남.
 - No SIMD
 - SIMD
 - SIMD + Multi Thread
 - CUDA
- 예제 코드 작성중 SIMD 코드 성능이 생각보다 높지 않음을 확인.
- 컴파일러가 만들어주는 코드가 예측과 많이 다름. 특히 SSE/AVX 버전 설정에 따라 많이 다름.
- 어디서 성능을 잡아먹는지 ASM코드로 분석을 해야겠다.

최적화 과정

- 어셈블리어로 작성 후 명령어를 바꿔보면서 성능 변화 측정.
- 의존성 있는 명령어들 사이사이에 의존성 없는 명령들을 끼워넣는 식으로 재배치하면서 성능 변화 측정.
- 상수 사용시 '메모리에서 로드 vs 레지스터만으로 생성'에 따른 성능 비교.
- '최대한 변수의 상태를 레지스터에 유지-레지스터 백업 증가' vs '레지스터 백업 감소-메모리 읽기/쓰기 증가' 성능 비교
- 어셈블리어 코드 작업후, 여기서 얻은 정보를 활용해서 Compiler Intrinsic코드도 수정 - 유의미한 성능 개선 확인.

작업 결과

```
-----  
IntersectTriangle_Normal : P(2596.2,-2400.0,-361.6)(t:0.47, u:0.02, v:0.50)[10000], 21.0162 clks elapsed.  
IntersectTriangle_SIMD : P(2596.2,-2400.0,-361.6)(t:0.47, u:0.02, v:0.50)[10000], 19.7958 clks elapsed.  
IntersectResult_AVX_ASM : P(2596.2,-2400.0,-361.6)(t:0.47, u:0.02, v:0.50)[10000], 17.7536 clks elapsed.  
IntersectResult_AVX_ASM_DevPark : P(2596.2,-2400.0,-361.6)(t:0.47, u:0.02, v:0.50)[10000], 17.9052 clks elapsed.
```

- 성능 순위
 1. 손으로 짠 어셈코드
 2. Compiler Intrinsic을 사용한 SIMD코드
 3. 순수 C코드(SIMD레지스터와 싱글명령어는 사용)
- Compiler Intrinsic을 사용한 SIMD코드도 처음엔 Non SIMD코드보다 느렸음.
 - 생성된 어셈블리어 코드를 보고 튜닝을 한 후에야 Non SIMD코드보다 빨라졌음.

- IntersectTriangle_Normal
 - 순수 C코드
 - XMM레지스터 사용, Vectorization 없음.
 - 21.0162 clks
- IntersectTriangle_SIMD
 - C코드 - Compiler Intrinsic 사용.
 - 수동 Vectorization
 - 19.7958 clks
- IntersectResult_AVX_ASM
 - 손으로 작성한 x64 asm by megayuchi
 - 수동 Vectorization
 - 17.7536 clks
- IntersectResult_AVX_ASM_DevPark
 - 손으로 작성한 x64 asm by DevPark
 - 수동 Vectorization
 - 17.9052 clks

최적화 결론

- dpps 명령이 많이 느림 - hadd나 shufps를 사용하는 편이 낫다.
- 수평합을 구할 때 hadd보다 shufps를 사용하는 쪽이 빠른 경우가 많음.
- 명령어 스케줄링을 적극적으로 사용할 것 - 의존성 있는 명령들 사이에 의존성 없는 명령을 끼워넣는 식으로 지연을 숨김.
- AVX2를 사용할 수 있다면 FMA명령 사용이 꽤 도움이 됨(내적, 외적 등)
- 12 Bytes float3타입을 읽고 쓸 때, 2+1로 처리해야 하는데 8바이트 읽기/쓰기에 movlps보다 movq가 빠름
 - Compiler Intrinsic으로 작성할 경우 일반적으로 movlps로 컴파일 되므로 어셈블리 코드 확인하면서 수정할 것.
- Compiler Intrinsic을 사용해서 명시적으로 Vectorization을 적용해도 세부 코드(movq vs movlps 등)에 따라 성능 차이가 많이 남.

공짜 점심은 없다!

- 생각보다 컴파일러가 Vectorization을 알아서 잘 해주지 않는다.
- 직접 Compiler Intrinsic으로 Vectorization코드를 작성한다 해도 빠른 코드가 나온다는 보장은 없음. 반드시 검증 필요.
- 같은 알고리즘이라 해도 명령어를 어떻게 사용하느냐에 따라 성능 차이가 크다.
- 컴파일러가 빠른 코드를 생성하도록 C코드를 튜닝해야 함.
- 손어셈블을 권장하는 것은 아니지만 손어셈블로 빠른 코드 작성할 정도의 기본은 되어야 Compiler Intrinsic을 잘 사용할 수 있음.
- ffmpeg에서는 arm코드까지 손어셈블로 짰다더라. 성능 부심 대단함.

AI에게 어셈코드 작성을 시켜볼까?

- 요새 들어 방송 들어와서 시비 거는 사람들이 많음.
 - AI가 코딩 잘 해주던데 그 기술(특정 도메인 특화 기술, API사용법 등) 왜 배워요? <- 실제 발언
- AI가 잘못된 코드를 뱉으면 어떻게 하려구요?
 - AI가 디버깅도 잘 해주던데요? <- 실제 발언
 - 디버깅은 AI가 해주는건데요? <- 실제 발언
- 최근 테스트와 맞물려 궁금증이 생김.
- AI가 고급언어로 예제 수준 코드는 잘 뱉어준다고 느낌.
- 그렇다면 asm코드를 짜보라고 하면 어떤 코드가 나올까?
- 손으로 최적화한 코드보다 빠를까?

테스트 대상 AI 서비스

- claude
- copilot(smart plus)
- chatgpt
- gemini-pro

이 C코드의 IntersectTriangle_Normal 함수를 m164로 어셈블 가능한 어셈블리어 코드로 바꿔줘. avx2까지 사용해서 최적화된 코드로 작성해줘.

```
BOOL IntersectTriangle_Normal(VECTOR3* pv3OutIntersectPoint, const VECTOR3* orig, const VECTOR3* dir, const VECTOR3* v0, const VECTOR3* v1, const VECTOR3* v2)
{
    BOOL bResult = FALSE;

    // Find vectors for two edges sharing vert0
    VECTOR3 edge1 = *v1 - *v0;
    VECTOR3 edge2 = *v2 - *v0;

    // Begin calculating determinant - also used to calculate U parameter
    VECTOR3 pvec = Cross(*dir, edge2);

    // If determinant is near zero, ray lies in plane of triangle
    float det = edge1.x * pvec.x + edge1.y * pvec.y + edge1.z * pvec.z;

    VECTOR3 tvec;
    if (det > 0)
    {
        tvec = *orig - *v0;
    }
    else
    {
        if (bCullBackface)
            goto lb_return;

        tvec = *v0 - *orig;
        det = -det;
    }
    if (det < 0.0001f)
        goto lb_return;

    // Calculate U parameter and test bounds
    *u = tvec.x * pvec.x + tvec.y * pvec.y + tvec.z * pvec.z;

    if (*u < 0.0f || *u > det)
```

필요한 데이터 타입 선언과 연산자 오버로딩, 인라인 함수들을 포함하여 빌드 가능한 cpp파일 한 개 분량으로 제공



No issues found



결과 요약

1. copilot - 어셈블 및 실행 가능(유일) - 잘못된 결과 출력
2. claude - 어셈블 불가 - 코드 고쳐서 어셈블 및 실행 - 크래시
3. gemini pro - 어셈블 불가 - 코드 고쳐서 어셈블 및 실행 - 고친 코드로 정상 작동 - Non SIMD/순수C코드와 비슷한 성능.
4. chatgpt 응답1- 어셈블 불가 - 코드 고쳐서 어셈블 및 실행 - 잘못된 결과 출력
5. chatgpt 응답2- 어셈블 불가 - 코드 고쳐서 어셈블 및 실행 - 크래시

테스트 결과(코드 수정 후)

```
=====
IntersectTriangle_Normal : P(2596.2,-2400.0,-361.6)(t:0.47, u:0.02, v:0.50)[10000], 21.0802 clks elapsed.
IntersectTriangle_SIMD : P(2596.2,-2400.0,-361.6)(t:0.47, u:0.02, v:0.50)[10000], 18.5344 clks elapsed.
IntersectResult_AVX_ASM : P(2596.2,-2400.0,-361.6)(t:0.47, u:0.02, v:0.50)[10000], 16.7684 clks elapsed.
IntersectResult_AVX_ASM_DevPark : P(2596.2,-2400.0,-361.6)(t:0.47, u:0.02, v:0.50)[10000], 17.1320 clks elapsed.
IntersectTriangle_CLAUDE : P(0.0,0.0,0.0)(t:0.00, u:0.00, v:0.00)[0], 0.0032 clks elapsed.
IntersectTriangle_CoPilot : P(0.0,0.0,0.0)(t:0.00, u:0.00, v:0.00)[0], 14.2360 clks elapsed.
IntersectTriangle_GeminiPro : P(2596.2,-2400.0,-361.6)(t:0.47, u:0.02, v:0.50)[10000], 20.7388 clks elapsed.
IntersectTriangle_ChatGPT_1 : P(0.0,0.0,0.0)(t:0.00, u:0.00, v:0.00)[0], 30.2604 clks elapsed.
IntersectTriangle_ChatGPT_2 : P(0.0,0.0,0.0)(t:0.00, u:0.00, v:0.00)[0], 0.0030 clks elapsed.
=====
```

- 정상적으로 어셈블 되는 케이스가 1개 뿐.
- 코드 수정 후에 정상작동한 케이스는 gemini-pro가 유일.
- gemini-pro가 작성한 코드의 성능은 순수 C로 작성한 Non SIMD코드 수준.

- IntersectTriangle_CLAUDE
 - Claude가 작성한 x64 asm
 - masm문법 오류
 - vmovss dword ptr[rsp+40h], xmm10 -> vmovss [rsp+40h], xmm10
 - vmovss xmm7, dword ptr [rel epsilon] -> vmovss xmm7, dword ptr [epsilon]
 - 실행하면 크래시
 - 스택프레임 생성 및 로컬 변수 액세스를 위한 rsp레지스터 사용이 잘못됐음.

- IntersectTriangle_CoPilot
 - CoPilot이 작성한 x64 asm
 - Vectorization 적용.
 - 요구한건 함수 1개인데 함수2개 만듦 - 코드 생성중 폐기된 코드가 남음.
 - 유일하게 코드 수정없이 어셈블 됨
 - 잘못된 결과 반환
 - 오작동으로 소요클럭 측정이 무의미함.

- IntersectTriangle_GeminiPro

- Gemini-Pro가 작성한 x64 asm

- Vectorization 적용.

- 명령어 사용법 오류

- vshufps xmm5, xmm5, 0C9h -> vshufps xmm5, xmm5, xmm5, 0C9h

- 코드 수정후 정상작동

- 1. 손어셈(megayuchi) – 16.7684 clks

- 2. 손어셈(DevPark) – 17.1320 clks

- 3. Compiler Intrinsic/SIMD – 18.5344 clks

- 4. **gemini-pro – 20.7388 clks**

- 5. 순수 C/Non SIMD – 21.0802 clks

- IntersectTriangle_ChatGPT_1
 - chatgpt가 작성한 x64 asm (응답 1)
 - Vectorization 적용.
 - 명령어 사용법 오류
 - vmovss xmm8, xmm7 -> vmovss xmm8, xmm8, xmm7
 - masn 문법 오류
 - vmulss xmm4, xmm0, [r13] -> vmulss xmm4, xmm0, dword ptr[r13]
 - 존재하지 않은 명령어 사용
 - vnegps xmm8, xmm8
 - 잘못된 결과 반환
 - 오작동으로 소요클럭 측정이 무의미함.

- IntersectTriangle_ChatGPT_2

- chatgpt가 작성한 x64 asm (응답 2)

- Vectorization 적용

- 명령어 사용법 오류

- vmovss xmm9, xmm8 -> vmovss xmm9, xmm9, xmm8

- masm 문법 오류

- vmovss [rax], xmm13 -> vmovss dword ptr[rax], xmm13

- vbroadcastss xmm15, [rax] -> vbroadcastss xmm15, dword ptr[rax]

- 실행하면 크래시

- 스택프레임 생성 및 로컬 변수 액세스를 위한 rsp레지스터 사용이 잘못됐음

자칭 AI전문가들의 주장

- 나는 기술 모르지만 자신 있어요.
- 내 로컬PC에 AI플랫폼 설치해서 내가 직접 학습시킴(뿌듯뿌듯)
- AI활용 정말 잘 함.
- 결론적으로 AI시대의 생존자가 될 것이다..
 -
 -
 -
- 정말 그럴까?

AI (에 시키는) 프로그래밍의 한계

- 언어논리에 기반하여 코드 작성
 - 언어논리에 기반하여 논리로 코드를 생성하므로 문장구조상 있을법한 API나 명령어를 지 멋대로 집어넣음.
 - 이걸 작성이라고 부르는게 맞나....코드 합성이지.
- 코드 작성 -> 테스트 -> 디버깅 의 절차를 따르지 않음.
 - 작동을 보장하지 않음
 - 프로그래밍은 글쓰기가 아니다.
 - 정상적인 인간이라면 코드를 짚으면 실행은 해본다.
 - 작동확인 안하고 코드 커밋하는 후임이 있다 생각해 보셈.
 - 제대로된 코드가 사람이 나오도록 유도를 한다면 더 나아지겠지만...
 - 코드의 잘못된 점을 짚어줄 사람은 어느 정도 스킬을 가지고 있어야 하나?

AI (에 시키는) 디버깅의 한계

- 일반적인 디버깅 프로세스
 1. 증상 재현
 2. 가설수립
 3. 가설검증(브레이크 포인트,로그수집,각종 관찰)
 4. 코드 수정
 5. 테스트
- AI는 일반적인 디버깅 프로세스를 수행하지 않음.
- 언어논리에 기반하여 코드 수정(디버깅이 아님)

결론

- 그 날이 언젠가 올 수 있겠지만 오늘은 아니다.
- AI가 도움이 될 수 있다.
- 기술을 알고 (시간절약 차원에서) 도움을 받는 것과 기술을 몰라서 전적으로 맡기는 것은 다르다.
- AI가 생성한 산출물이 의도한 방향으로 가게 하려면 지시하는 인간이 기술을 잘 알아야 한다.
- AI를 활용하는 측면에서도 결국은 도메인 스킬이 뛰어난 전문가가 유리하다.
- 기술 몰라서 ai한테 의존하는 인간들은 어차피 ai시대에서도 경쟁력이 없다.

하지만 어차피 디스토피아 확정

- AI에 대한 맹목적인 믿음으로 인력들을 모두 AI로 대체(대량해고)
 - 결과가 나쁘지 않았다
 - 그나마 남은 인력들도 해고 – 일자리 증발
 - 결과가 나빴다
 - 회사가 망한다 – 일자리 증발